Sorting networks and fixed machines

Mathematical overview | Draft and unpiloted | 3 October 2026

The mathematical core

A comparator replaces its two input values by their minimum and maximum, with the minimum in the upper lane (the lane 1 side). A network is a fixed sequence of such comparisons, independent of the input. A sorter must finish in nondecreasing order for every input, including repeated values.

Zero–one principle: A fixed n-lane comparator network sorts every numerical input if and only if it sorts all 2^n binary inputs. Thresholding each value to 0 when it is at most t and to 1 otherwise commutes with every comparator. An inverted output pair can therefore be turned into a binary inversion.

Exact small minima: Three lanes need three comparators; four lanes need five. The guide gives attaining networks. If only neighboring lanes may be compared, four lanes need six. For unrestricted networks, m swap/no-swap decisions give at most 2^m histories, while sorting n distinct ranks requires at least n! different histories. The neighboring-lane bound instead counts the six inversions of 4321.

What the activity develops

K–1: Run a fixed compare-exchange action with dot or length cards, find a failing start, and build or repair small machines. The shared launch models the compare-exchange action before a short two-bar run.

Grades 2–3: Organize complete start collections, include every binary weight, and distinguish a working construction from one proved minimal.

Grades 4–5: Connect threshold conversions to binary certification and reversible swap records to lower bounds. Small examples introduce each new notation without giving a complete sorter.

From trials to explanation

One failing input disproves a sorter claim. Many successful trials suggest a conjecture; exhaustive binary coverage plus the zero–one argument certifies all numerical inputs. A minimum also needs a lower bound. Detailed constructions and proofs follow on printed pages 3 and 9–11.

Sorting machines
Facilitator guide

Week 23. Draft and unpiloted. Unscheduled library slot. This guide accompanies the F23-K-v4, F23-23-v3 and F23-45-v3 student packets (6, 6 and 7 pages). It is an adult key, not a lesson already tested with children. Upper pages include small notation examples.

Purpose and quick preparation

Children build a fixed machine, challenge it, and decide what counts as enough evidence. A short counterexample can defeat a machine; certifying every input and proving a minimum require different arguments. The concrete action is compare-exchange, with smaller values going toward lane 1.

Table kit: week-23-materials.pdf (print at 100%; physically unrehearsed). For each of three tables: a four-lane mat made from two taped Letter sheets (lanes 45 mm apart, 38 mm start and finish boxes); one card-stock sheet of 35 mm cards, with dot cards 0, 0, 1, 1, 2, 3 and 4 for K-1 and number cards 1-4 twice plus four 0s and four 1s for the older tables; one card-stock sheet of 14 bars (8 join neighboring lanes, 4 skip a lane, 2 join lanes 1 and 4) and a time strip. Add a marker and blank paper.

Lanes are numbered 1-4 from top to bottom and every bar has exactly two endpoint dots. A long bar may cross another lane without comparing it. Leave a card's width between bars; the joined mat has room for seven. Slide the time strip left to right as the cards move. Do not use the small printed input boxes as card holders: they are recording diagrams, much smaller than the cards.

Before children arrive, about 40 minutes the first time: print the kit (four sheets per table) and the chosen student pages single-sided on Letter at 100%; cut each table's cards and bars (about 10 minutes per table) and tape its mat; rehearse input 312 on bars (1,2), (2,3); set aside answer diagrams. The kit is reusable. Read the applicable key pages. Supply many spare bars so equipment does not announce the optimum. Read directions aloud as needed; do not require reading or writing to participate.

A short shared launch

Give 3 minutes of free handling. Then demonstrate only one bar with cards 3 and 1: both arrive, compare, send 1 above and 3 below. Repeat with already ordered cards and equal cards. Add a third lane; point out that it is untouched. Say: "The bars stay in the same places for every start. Can a start make the machine finish out of order?" Run one two-bar example: start 312 on bars (1,2), (2,3), which sorts. Write the start beside the start boxes and the finish beside the finish boxes, one number per lane from lane 1 down (3, 1, 2 and 1, 2, 3), the way the older tables record starts. Then let groups begin. Do not reveal a complete sorter in the launch.

Ground rules: move all cards past the next bar together; never reorder by hand elsewhere, choose the next comparison after looking at values, or reverse a comparator. Ties can stay. **Partners keep the rule:** one child chooses the start; the partner slides the time strip one bar at a time, moves the cards behind it and says "swap" or "stay" at each bar; they switch jobs for each start. At the 4-5 table a third child referees. Reading, motor speed and ability to copy a diagram are not mathematical gates.

An hour with flexible routes

Time	Common plan
0-5 min	Arrival and about five minutes of running around.
5-12 min	Handle materials, then shared one-bar launch.
12-27 min	One substantial opening task at each table; preserve time for failed attempts.
27-30 min	Optional movement reset: three volunteers model a single comparator.
30-50 min	Continue the same investigation or choose the next ready task.
50-57 min	Compare one working machine and one failing start across tables.
57-60 min	Photograph a construction, name an open question, and reset materials.

K-1 route: Problems 1-2 are a full core. Offer Problem 3 to compare candidates or Problem 4 when repeated cards are familiar. Problems 5-6 are reserves. A child may demonstrate a failure or make a complete six-start collection orally.

Grades 2-3 route: Problems 1-3 are a repair and minimum route. Problems 4-5 form a separate binary-certification route. Problem 6 introduces a new constraint and a new lower-bound argument; save it until children can track all four cards reliably.

Grades 4-5 route: start with Problems 1-2. Choose either Problems 3-5 (threshold certification) or Problems 6-7 (records and minima). Covering both proof routes in one hour is optional; retain a live question for another meeting.

Prerequisite gates and adult responses

Concrete entry: compare two lengths or dot quantities and track three lanes. The adult moves cards or records starts on request. If children forget a rule, rerun a single bar, then return to their own machine. Offer a pointer, larger cards, tactile lane edges and spoken lane names; use position and labels, not color alone.

Case-coverage gate: distinguish a starting arrangement from its final order; arrange six starts without omissions. A photographed set of six physical starts is as valid as a written list. For binary work, sort test cards by number of 1s, only if organization has become a genuine obstacle.

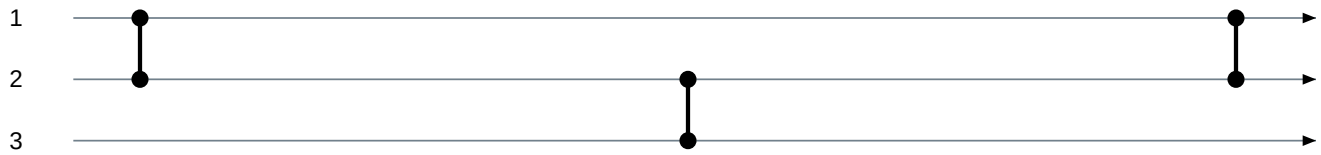
Proof gates: thresholds need comparison with a chosen number, including negative numbers and equality. History counting needs two choices per bar and reversible token moves. Inversion counting needs relative order of pairs, not multiplication fluency. If those gates are absent, keep the full adult argument available without making it a child performance requirement.

Hold hints: first ask what the child has tried and wait. Then offer the first hint in the key. Give the second only after another attempt. A counterexample is a complete disproof, but many successes are not automatically a universal proof. A successful construction is not automatically minimal.

Common confusions: the upper lane is the one nearer lane 1, higher on the page; a crossed lane without an endpoint dot is untouched; five bars are five comparisons even if disjoint bars share a stage. A different bar order can change the machine. Accept any valid construction with a complete justification, not only the pictured one.

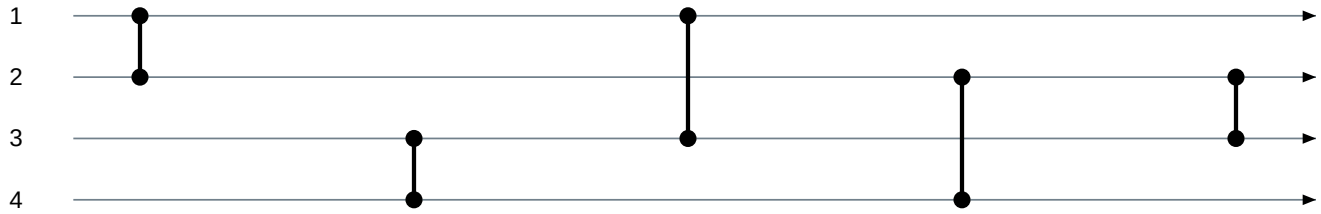
Adult reference Working machines and guarantees

S3 Three bars: (1,2), (2,3), (1,2)



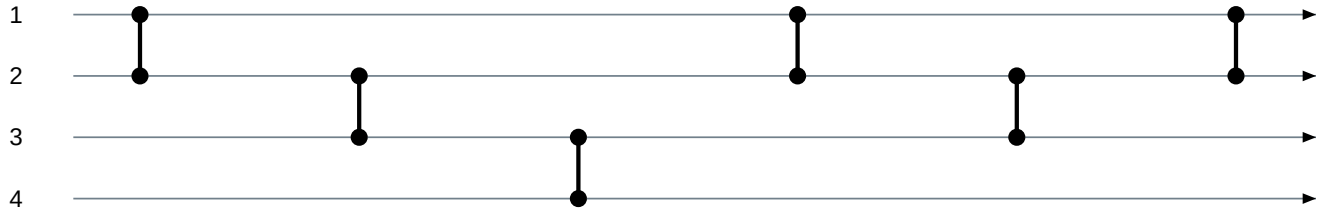
Why S3 works for all values. The first bar orders lanes 1 and 2. The next sends the largest of all three to lane 3. The last orders the two remaining values. This includes ties. The alternative Q = (1,3), (1,2), (2,3) works too: its first two bars put the minimum in lane 1, and the last orders lanes 2 and 3.

S4 Five bars: (1,2), (3,4), (1,3), (2,4), (2,3)



Why S4 works. After the first two bars write the values as a,b,c,d with a <= b and c <= d. Comparing a with c puts the global minimum in lane 1. Comparing b with d puts the global maximum in lane 4. The last bar orders the two middle values. Neither cross-lane bar touches the other pair of endpoints.

N4 Neighbor-only: (1,2), (2,3), (3,4), (1,2), (2,3), (1,2)



Why N4 works and why six is optimal. The first three comparisons send the maximum to lane 4. The next two send the maximum of the remaining three to lane 3; the last orders lanes 1 and 2. In 4321, all six pairs are reversed. Swapping neighboring cards changes only the relative order of that one pair. To reach 1234, every pair must cross; each bar can swap at most one pair. Therefore at least six bars are required. A no-swap bar removes no reversed pair.

Notation in this key. A string such as 231 gives lane values from top to bottom, never left to right across comparator times. Read (1,3) as a bar whose only endpoints are lanes 1 and 3. The diagrams are adult answer references, not a proposed replacement for discovery.

K-1 Problems 1 to 3

Problem 1 Test all six starts

Solution. The machine is T = (1,2), (2,3). Circle 123, 132, 213 and 312. Do not circle 231 or 321: both finish 213. The six pictures run in that order across the first row and then the second row. See the complete table below.

Hold these hints. First: "Have we tried the starts with 1 at the bottom?" Later: hold the first bar fixed and follow just those two starts.

Readiness and stopping point. Comparing three quantities is enough. Adult records are fine. Stop when every pictured start has a tested outcome.

Start	After T	After S3
123	123	123
132	123	123
213	123	123
231	213	123
312	123	123
321	213	123

Problem 2 Invent the smallest three-lane sorter

Solution. Three bars suffice: use S3 on guide page 3, or Q. Fewer do not: two bars have at most four swap/stay records, while there are six distinct starts. The same record reverses from 123 to only one start (full argument on page 11). Alternatively physically inspect all nine two-bar lists using page 6. Zero- and one-bar machines are included, because padding them with repeated bars cannot spoil an already sorted output.

Hold these hints. First: "Which starts still fail on your machine?" Later: try a final bar that repairs 213. Hold back the complete S3 until children have an invention attempt.

Readiness and stopping point. Making and testing a three-bar construction is a worthwhile stopping point. The minimum proof may be adult-supported; do not confuse failure to invent a two-bar sorter with proof of impossibility.

Problem 3 Challenge two three-bar machines

Solution. Upper machine S3 sorts every start, so its failure list is empty. Lower machine (1,2), (1,2), (2,3) fails exactly on 231 and 321, finishing 213. Its second bar does nothing: the same pair is already ordered, so it acts just like T. This accounts for all six starts. Under each machine the six pictures run 123, 132, 213, 231, 312, 321 from left to right, as in Problem 1, so children circle nothing under the upper machine and the fourth and sixth pictures under the lower one.

Hold these hints. First: "What can happen at the repeated bar?" Later: compare the state just after its first and second bars.

Readiness and stopping point. Use physical cards and the six-start list. Accept pointing to a final inverted pair as an explanation; keep the notion of a fixed machine explicit.

K-1 Problems 4 to 6

Problem 4 Separate the two repeated-card sets

Solution. For the left set 001 use T = (1,2), (2,3). It sends the only 1 to lane 3 from any starting lane. It fails on right-set start 110: 110 -> 110 -> 101. For the right set 011 use R = (2,3), (1,2). It sends the only 0 to lane 1. It fails on left-set start 100: 100 -> 100 -> 010. These meet both parts of each request: always on one set and sometimes not on the other.

Hold these hints. First: "Which single card needs to get all the way to one end?" Later: try two bars in the opposite order.

Readiness and stopping point. Children need to treat the blank dot card as zero and distinguish one 1 from two 1s. Test each of the three arrangements of each set; equal cards need not be distinguished.

Machine	All starts it must sort	Counterexample on other set
T: (1,2), (2,3)	001, 010, 100 -> 001	110 -> 101
R: (2,3), (1,2)	011, 101, 110 -> 011	100 -> 010

Problem 5 Add the missing last bar

Solution. Top: append (1,2) to (1,2), (2,3). Middle: append (2,3) to (2,3), (1,2). Bottom: append (2,3) to (1,3), (1,2). These are the unique successful final bars. In the top machine the maximum is already in lane 3; in the other two the minimum is already in lane 1. Sort the remaining pair. Each failing prefix really leaves that pair reversed on some start, so comparing a different pair does not always repair it.

Hold these hints. First: "Which card already ends in the right lane every time?" Later: point to the two lanes still reversed on one failed test.

Readiness and stopping point. Read the bottom long bar carefully: lane 2 is not an endpoint. Test all six starts after a repair, not just the example used to invent it.

Problem 6 Try four cards

Solution. Five bars suffice with S4 on page 3. Its minimum/maximum proof certifies all orders. Four bars cannot suffice: 24 distinct starts would need 24 distinct swap/stay records, but four bars allow at most 16 (page 11). The adult may count the 24 starts as four choices for lane 1, then three, then two, then one.

Hold these hints. First: "Can you order two pairs before joining them?" Later: "Where could the smallest and largest cards be?"

Readiness and stopping point. This is an optional four-lane extension, not an expected K-1 finish. A correct six-bar neighbor machine is progress; the five-bar optimum and universal lower bound can remain an adult explanation. Do not claim a child discovered or proved them without observing it.

Grades 2-3 Problems 1 and 2

Problem 1 Collect every successful start

Solution. Top T sorts 123,132,213,312; it fails 231,321 -> 213. Middle R sorts 123,132,213,231; it fails 312,321 -> 132. Bottom Q = (1,3),(1,2),(2,3) sorts all six. The first two bars of Q put the minimum in lane 1; its last bar orders the remaining pair.

Hold these hints. First: "How will you know none of the six starts is missing?" Later: fix one card in lane 1 and swap the other two, then change the first card.

Readiness and stopping point. A complete physical collection with an adult scribe is enough. The order of successful tests is immaterial.

Problem 2 Three bars and a lower bound

Solution. S3 or Q gives a three-bar construction. The table below gives a failing input for every possible two-bar machine. There are exactly three allowed lane pairs and three choices for the next bar, giving all nine lists. Thus no two-bar sorter works. Padding a shorter proposed sorter with extra comparators would keep its sorted outputs sorted, so this also excludes zero or one bar. The history argument on page 11 is an alternative.

Hold these hints. First: "How many choices are there for one bar?" Later: make three piles of candidate lists, one for each first bar.

Readiness and stopping point. Try the tangible nine-list proof before powers of two if needed. Distinguish all two-bar lists from merely removing a bar from one favorite sorter.

First bar	Second bar	One failing start	Final order
(1,2)	(1,2)	132	132
(1,2)	(1,3)	132	132
(1,2)	(2,3)	231	213
(1,3)	(1,2)	132	132
(1,3)	(1,3)	132	132
(1,3)	(2,3)	213	213
(2,3)	(1,2)	312	132
(2,3)	(1,3)	213	213
(2,3)	(2,3)	213	213

Do not hand out the table first. It is an adult certificate and a way to check a child-generated list. One row is a suitable held scaffold when the child wants a systematic impossibility argument.

Grades 2-3 Problems 3 and 4

Problem 3 Repair or rule out a one-bar repair

Solution. Top: append (2,3), giving S4. Middle: append (2,3) to (1,3),(2,4),(1,2),(3,4). After the first two comparisons, minima of the two original pairs occupy lanes 1,2 and maxima occupy lanes 3,4; the next two put the global minimum in 1 and maximum in 4. The final bar orders the middle. Bottom: impossible. Input 2413 finishes 2143 under (1,2),(3,4),(1,4),(2,3). Both disjoint pairs 2,1 and 4,3 are reversed, and all four positions are wrong. One bar can change only two positions, so cannot produce 1234. This one input defeats every possible appended bar.

Hold these hints. First: "What remains wrong just before the blank last bar?" Later: for the bottom, seek a finish with four positions wrong.

Readiness and stopping point. A failed attempted repair is not enough for impossibility. The witness 2143 plus the two-position argument is enough. Long bars compare endpoints only. Certifying a repair by tests takes all 24 starts: split them by top card (six each) and pool them across children, or argue as above that the smallest card reaches lane 1 and the largest lane 4 before the last bar.

Problem 4 An incomplete binary certificate

Solution. All six arrangements of 0011 are 0011,0101,0110,1001,1010,1100. The printed machine W = (1,2),(1,3),(2,3),(2,4) sends each to 0011. It does not sort every binary start. Exactly these fail: 0010,0100,1000 finish 0010; 1110 finishes 1011. The first three have one 1; the last has three 1s. Testing only a fixed number of 1s misses them.

Hold these hints. First: "What kinds of starts have we not used at all?" Later: use one 1, then move it through the four lanes.

Readiness and stopping point. Children should understand that equal cards create fewer different value arrangements than labeled-card permutations. Stop once the six-start claim and a counterexample to the stronger claim are both clear.

Number of 1s	All binary starts in this class	W result
0	0000	Sorted
1	0001, 0010, 0100, 1000	Only 0001 sorts
2	0011, 0101, 0110, 1001, 1010, 1100	All sort
3	0111, 1011, 1101, 1110	Only 1110 fails
4	1111	Sorted

Teaching choice. Let children find a counterexample before supplying this class-by-class audit. The distinction is substantive: a full certificate needs every binary input, not many trials of the one pictured multiset.

Grades 2-3 Problems 5 and 6

Problem 5 Build and certify a binary sorter

Solution. Use S4 (five bars) or N4 (six neighbor-only bars), both on page 3. The complete S4 audit below checks all 16 starts once; there are 2 choices in each of 4 lanes. Each output has the same number of 1s as its input and is ordered. Problem 5 asks for a full record, not an optimum. A different successful fixed construction is valid.

Hold these hints. First: "How could you group starts so a missing one is visible?" Later: arrange by number of 1s, or choose the first bit and list every three-bit ending. Held question, after a complete record: "Does your machine sort every order of 1, 2, 3, 4? Can a machine sort every 0-1 start and still fail one?" It cannot: this is the zero-one principle (Grades 4-5 Problem 4, page 10), so their 16 tests already cover all 24 orders.

Readiness and stopping point. Keep the audit manageable with photographs, stamps or adult transcription. Repeated random starts cannot establish completeness.

Start	Output	Start	Output
0000	0000	1000	0001
0001	0001	1001	0011
0010	0001	1010	0011
0011	0011	1011	0111
0100	0001	1100	0011
0101	0011	1101	0111
0110	0011	1110	0111
0111	0111	1111	1111

Problem 6 The neighbor-only optimum

Solution. Exactly six bars. N4 gives the construction and the proof on page 3. For a lower bound start 4321: the six reversed pairs are 4/3,4/2,4/1,3/2,3/1,2/1. To sort, each pair must reverse relative order. A neighboring swap crosses only one pair; it cannot change any other pair's relative order. Thus at least six swaps, and at least six bars, are needed. This is stronger than the unrestricted five-bar lower bound because the neighbor restriction matters.

Hold these hints. First: "What must happen to each pair in 4321?" Later: place a mark for each reversed pair and see how many disappear in one adjacent swap.

Readiness and stopping point. Require neighboring endpoints on every bar. Trace pairs with labeled strips if inversion counting is unfamiliar. A valid six-bar sorter plus this pair-crossing explanation completes the task.

Reverse-input trace for N4: 4321 -> 3421 -> 3241 -> 3214 -> 2314 -> 2134 -> 1234. Every bar swaps on this input. Independently checked: all 24 distinct starts and all 16 binary starts pass N4; none of the 243 length-five neighbor-only lists sorts all distinct starts.

Grades 4-5 Problems 1 to 3

Problem 1 Distinct and repeated values

Solution. Upper machine T fails exactly on 231 and 321 among distinct ranks; both finish 213. It sorts 113,131,311, all to 113. For 133 it sorts 133 and 313, but 331 finishes 313. Lower machine Q sorts every one of these inputs by the minimum-then-last-pair argument on page 3. Distinct-rank tests alone are not a written explanation of repeated-value behavior unless an order-preserving argument is supplied.

Hold these hints. First: "Which distinct-rank failure might survive when two cards become equal?" Later: compare 231 with 331 and with 311.

Readiness and stopping point. Track equal values without pretending identical cards yield different value arrangements. A small explicit test list is valid here.

Problem 2 Construct a four-lane sorter

Solution. S4 uses five bars and its general proof is on page 3. The exact minimum is five, by the 24-input versus 16-history lower bound on page 11. If a child finds N4 first, keep it as a correct six-bar machine and look for a shorter one; do not imply the local neighbor-only bound applies when long bars are allowed.

Hold these hints. First: "Can you first sort two pairs?" Later: "Can the smallest and largest be settled before the middle pair?"

Readiness and stopping point. The proof of optimality can wait until Problem 7. Preserve a photograph of each original construction before editing.

Problem 3 Every threshold row

Solution. The rows and exact threshold ranges are below. With the rule value <= t gives 0, equality belongs to the interval on its right. Equal values cross the threshold together.

Start	Threshold range	Bit row
(-2,8,5,8)	t < -2; -2 <= t < 5; 5 <= t < 8; t >= 8	1111; 0111; 0101; 0000
(6,2,9,4)	t < 2; 2 <= t < 4; 4 <= t < 6; 6 <= t < 9; t >= 9	1111; 1011; 1010; 0010; 0000
(4,4,4,4)	t < 4; t >= 4	1111; 0000

Commutation argument. For two values, either both are at most t (both bits 0), both exceed t (both 1), or one lies on each side (comparison gives 0 above 1). These exhaust the cases. Comparing numbers then replacing them therefore gives the same bits as replacing first and comparing the bits.

Hold hints. First: move t along a number line. Later: only passing one of the displayed distinct values can change a bit. **Gate:** read negative numbers and inclusive inequality; offer a number line and oral description. Stop when all rows and the three-case comparator argument are accounted for.

Grades 4-5 Problems 4 and 5

Problem 4 Why binary tests cover all numbers

Solution. No. Choose any t with 4 <= t < 9, for example t=4. Thresholding the alleged output makes lane 2 a 1 and lane 3 a 0. By Problem 3, thresholding commutes with every comparator, hence with the complete fixed list, one bar at a time. The thresholded input is a binary start that would end unsorted, contradicting the premise. More generally, any unsorted numerical output has an earlier value x larger than a later y. Choose t=y. The same argument yields an earlier 1 and later 0. Therefore sorting every binary input guarantees sorting every numerical input, including repeated values.

Hold these hints. First: "Choose a threshold separating 4 and 9." Later: compare the two routes through one bar before extending to the whole machine.

Readiness and stopping point. The key gate is the one-bar commuting argument, not symbolic proof language. Children may draw two paths that meet at the same bit output. The network must remain fixed for this reasoning.

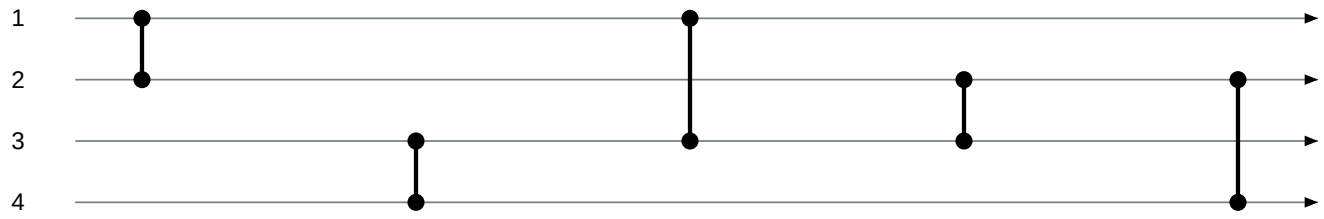
Problem 5 Which five-bar machine really sorts

Solution. Upper is S4 and sorts all numerical inputs. One complete justification is its 16-row audit on page 8 combined with Problem 4; another is the minimum/maximum proof on page 3. Lower has bars (1,2),(3,4),(1,3),(2,3),(2,4). It fails on binary inputs 0100 and 1000, both ending 0010. Thus it cannot sort every numerical input; a single binary counterexample suffices, including a repeated-value input. All its other binary starts sort.

Hold these hints. First: "Does moving the middle comparison earlier leave a new problem later?" Later: try a single 1 in lane 1 or lane 2.

Readiness and stopping point. Do not treat identical bar sets as identical networks: their order matters. After finding a failure, stop testing the failed candidate if the requested decision is already settled.

Lower printed candidate Same pairs, harmful order



Explicit failing trace: 1000 -> 0100 -> 0100 -> 0100 -> 0010 -> 0010. The fourth bar moves the 1 to lane 3, and the last bar never touches lane 3. This is a useful way to check a child's counterexample without giving the answer in advance.

A worthwhile stopping point

A group that understands why sixteen deliberate tests certify infinitely many numerical inputs has completed a substantial proof investigation. It need not also finish the optimality route. Invite a child to explain what a threshold does with a new repeated-value example, then preserve the unfinished minimum question for later.

Grades 4-5 Problems 6 and 7

Problem 6 All swap and stay records

Solution. Read S as a swap and N as no swap, from left to right. These are every record and every start producing it; the six permutations exhaust the input set.

Machine	Record	All starts	Final order(s)
T: (1,2),(2,3)	NN	123	123
T	NS	132, 231	123, 213
T	SN	213	123
T	SS	312, 321	123, 213
S3: (1,2),(2,3),(1,2)	NNN	123	123
S3	NSN	132	123
S3	SNN	213	123
S3	NSS	231	123
S3	SSN	312	123
S3	SSS	321	123

Why two different starts cannot both sort with the same record. Fix the lane pairs and record. Every S swaps a specified pair of positions and every N leaves them alone; together these are one fixed, reversible permutation of tokens. Starting at sorted 123 and undoing this record in reverse order recovers one unique start. Two starts can share a record, as T shows, but at most one finishes sorted.

Hold hints. First: group trials by their S/N string. Later: run one record backward from 123. **Gate:** distinct cards and reversible swaps; do not apply this count to indistinguishable repeated-value arrangements.

Problem 7 Exact unrestricted minima

Solution. Three lanes need exactly 3 bars; four lanes need exactly 5. S3 and S4 attain them (page 3). A k-bar network has at most 2^k records, because each bar contributes S or N. Some strings may never occur, which only lowers the count. Every distinct input order needs its own successful record by Problem 6. For three cards, 3 x 2 x 1 = 6 starts exceed $2^2 = 4$ records. For four cards, 4 x 3 x 2 x 1 = 24 starts exceed $2^4 = 16$ records. Therefore two or four bars respectively cannot work, and neither can shorter lists.

Hold hints. First: compare the count of starts with possible records. Later: build the two-choice record tree for two bars and extend it. **Gate:** counts of choices and the one-record argument must both be understood. Merely naming 6 > 4 and 24 > 16 leaves the crucial connection unstated. Count bars, not parallel stages.

Extensions and responsible reuse

Depth extension, after a successful sorter. Which bars can run together? S4 has the three stages {(1,2),(3,4)}, {(1,3),(2,4)}, {(2,3)}. A lane cannot take part in two simultaneous comparisons. Three stages are optimal on four lanes: at most two disjoint bars fit in one stage on four lanes, so two stages contain at most four bars, fewer than the five-bar lower bound. Keep this as a separate question from bar count.

Neighbor extension. For n distinct cards in reverse order, all n(n-1)/2 pairs must cross using adjacent swaps. The fixed repeated left-to-right pass construction uses exactly (n-1)+(n-2)+...+1 bars and sorts by fixing the largest remaining card at each pass. Thus the neighbor-only optimum is n(n-1)/2. Invite children to establish n=5 before generalizing.

Order-preserving relabeling. Replace 1,2,3,4 by 10,20,30,40 or four increasing lengths. Every comparison decision is unchanged. Ask what fails if the relabeling reverses just two ranks. This develops the threshold idea without requiring negative numbers.

Sources and evidence limits

Mathematical source: University of Liverpool, COMP308, Lecture 17, "Sorting networks," PDF pp. 1-2, comparator definition and zero-one principle. Verified 2026-10-03: cgi.csc.liv.ac.uk/~igor/COMP308/files/Lecture17.pdf. The exact small constructions, repairs, histories and minima in this guide are independently derived and exhaustively checked; the lecture is not being cited as their classroom test.

Teaching source: Laura Givental, Maria Nemirovskaya and Ilya Zakharevich, *Math Circle by the Bay*, preface printed pp. viii-x (local PDF pp. 9-11): substantial themes across levels, manipulatives and dialogue, clear accessible tasks, and adaptable depth and pace. Read directly. Our minute menu, particular hints and readiness gates are proposed applications, not claims tested in that book or with this group.

Local evidence: README.md and plans/week-01-classroom-review.md record the organizer's need for a shared concrete launch, enough independent work, and representations introduced when useful. Those observations concern earlier activities. They support the format choice, not an assertion that sorting networks will engage every child.

Verification and after-session notes

checks.py independently enumerates all 6 and 24 distinct starts, all 16 four-bit starts, all 9 two-bar three-lane lists, all 1,296 four-bar unrestricted lists and all 243 five-bar neighbor lists. It validates repair uniqueness and the exact histories. checks.json preserves results. The human arguments above explain why the claims hold; exhaustive checks are an additional error detector.

Record which problems and examples were actually attempted, which rules needed repair, which arguments children supplied, and where a hint changed the activity. Distinguish observations from guesses about causes. Do not turn this prepared guide into a claim of classroom success or schedule it merely because it is numbered Week 23.